1 Part 3: Context Window Extension

1.1 Overview and Findings

Context window extension trains a language model at a short sequence length, then resumes training at a longer sequence length. The hypothesis is that most language knowledge is learned during the short-context phase, and extension mainly teaches the model to attend over longer positions—making it a cheaper alternative to training at full context from the start.

This is well-established in the literature. GrowLength [1] demonstrates progressive context growth (e.g. 128→256→512→1024) during pretraining, finding that short-context phases contribute the bulk of capability and that extension mainly adapts attention patterns to longer positions [2, 3].

We train "picochat"—a smaller nanochat configuration—at sequence length 512, extend to 2048, and compare against training at 2048 from scratch. Our key findings:

- Context extension mechanically works: the extended model handles long positions better than the short-only model (mean CE 4.19 vs 4.34 beyond position 512).
- The degradation beyond the training boundary is gradual (+0.35 nats), not catastrophic—RoPE extrapolation at 4× is within tolerance at this model scale.
- The cheap path (extension) nearly matches the expensive path (full training) on BPB (1.0540 vs 1.0499) and CORE (0.0766 vs 0.0789), but leaves a 28% gap on PG19 perplexity (73.55 vs 57.40) due to unmitigated forgetting.

1.2 Method and Justification

1.2.1 Model Configuration

Picochat uses nanochat's depth-6 configuration:

Parameter	Value
Depth	6
Model dim	384
Attention heads	3
Head dim	128
Window pattern	SSSL
Total parameters	∼136M
Scaling parameters	∼35.8M

Depth 6 is the smallest nanochat configuration with 3 attention heads (depth 4 has only 2). Nanochat's own `runcpu.sh` uses depth 6 for demos.

1.2.2 Sequence Length

The target of 2048 is nanochat's default `max_seq_len` and the assignment requirement. No RoPE interpolation is needed—we are returning to the native context length. The question is what short

length to start from.

We chose 512, grounding the decision in two sources:

1. **BabyLM** [4] tests 125M-parameter models across {64, 128, 256, 512, 1024, 2048, 4096, 8192} and recommends 512 as "a safe and efficient baseline across both architectures."
2. **GrowLength** [1] validates 2× per-stage jumps as the established baseline for progressive context growth.

This places our 4× jump (512→2048) as a deliberate sweet spot. A 2× jump (1024→2048) is already validated by GrowLength as their baseline—repeating it adds no new insight—and offers only modest compute savings since attention is a small fraction of total FLOPs at this model scale. An 8× jump (256→2048) pushes further into territory that GrowLength warns against: they find that "substantial difference between consecutive training window sizes can lead to dramatic loss rising," though their experiments are on multi-stage schedules, not single-stage jumps. Our 4× is a moderate extrapolation beyond the validated 2×—large enough for meaningful savings, small enough to stay closer to the safe range.

1.2.3 Data Portion

The token budget of ∼376M is 0.38% of the FineWeb-edu 100B-token corpus. This is the compute-optimal amount for a 35.8M scaling-parameter model under nanochat's 10.5× data ratio, which follows Chinchilla-style scaling laws [5]. We use nanochat's default ratio because the codebase's hyperparameters (LR, batch size, weight decay) are co-tuned for it.

This gives a three-stage experiment:

Stage	Strategy	seq_len	Steps	Purpose
1	Train from scratch	512	1433	Short-context pretraining
2	Resume from Stage 1	2048	500 (1433→1933)	Context extension (cheap path)
3	Train from scratch	2048	1433	Full-context baseline (expensive path)

Stages 1 and 3 use nanochat's auto-computed iteration count (`target_tokens / total_batch_size` = 375,732,000/262,144 ≈ 1433). The literature provides no quantitative recovery data for single-stage context jumps, so Stage 2 uses a generous 500 extension steps with `save_every=50` to observe the actual trajectory at high resolution (10 intermediate checkpoints). In hindsight, recovery largely plateaus by step ∼1600 (∼170 steps in), so fewer steps would have sufficed—but this was unknown beforehand. On resume, nanochat recalculates the LR schedule for `num_iterations=1933`, producing a deliberate rewarm from ≈0 to ≈0.52 of peak LR at step 1433—analogous to ProLong's approach [3].

1.2.4 Custom Evaluation

We designed a per-position cross-entropy evaluation to directly measure whether the model handles positions beyond its training length:

- **Dataset**: PG19 test split—avoids contamination (nanochat trains on FineWeb-edu) and is standard in context extension literature [6, 7, 8].
- **Bucketing**: 128-token windows (16 buckets for 2048 tokens). Each bucket reports mean cross-entropy across all documents.
- **Sample**: All 100 documents in PG19 test split, each truncated to 2048 tokens.

We also run nanochat's built-in evaluations:

- **BPB** (bits per byte): Overall language modeling quality on FineWeb-edu validation.
- **CORE** (22 tasks): General task capability. At ∼136M params, most tasks are near random—HellaSwag, PIQA, ARC-Easy, and BoolQ are the most informative. The appendix lists all 22 per-task scores.

1.2.5 Checkpoints

Checkpoint	Model Tag	Step	Description
Short	pico-short	1433	End of Stage 1 (seq_len=512 only)
Extended	pico-short	1933	End of Stage 2 (512→2048 extension)
Full	pico-full	1433	End of Stage 3 (2048 from scratch)

Additionally, 10 intermediate checkpoints during extension (steps 1450–1900, every 50 steps) were evaluated with both custom eval and CORE to track recovery dynamics.

CORE was not run on the short checkpoint (pico-short@1433) because nanochat's RoPE positional encoding at seq_len=512 cannot process the 2048-token CORE prompts.

1.2.6 Compute Specification

All training and evaluation ran on A100-80GB GPUs via Modal. We chose this GPU for two reasons. First, the full batch at `seq_len=2048` (∼262,144 tokens) fits in 80 GB, allowing one forward pass per optimizer step with no gradient accumulation. Second, nanochat's built-in CORE evaluation at 2048-token prompts exceeded 40 GB VRAM, ruling out the A100-40GB. All stages use nanochat's default learning rate schedule (warmup ratio 0, warmdown ratio 0.5).

1.3 Results

1.3.1 Per-Position Cross-Entropy

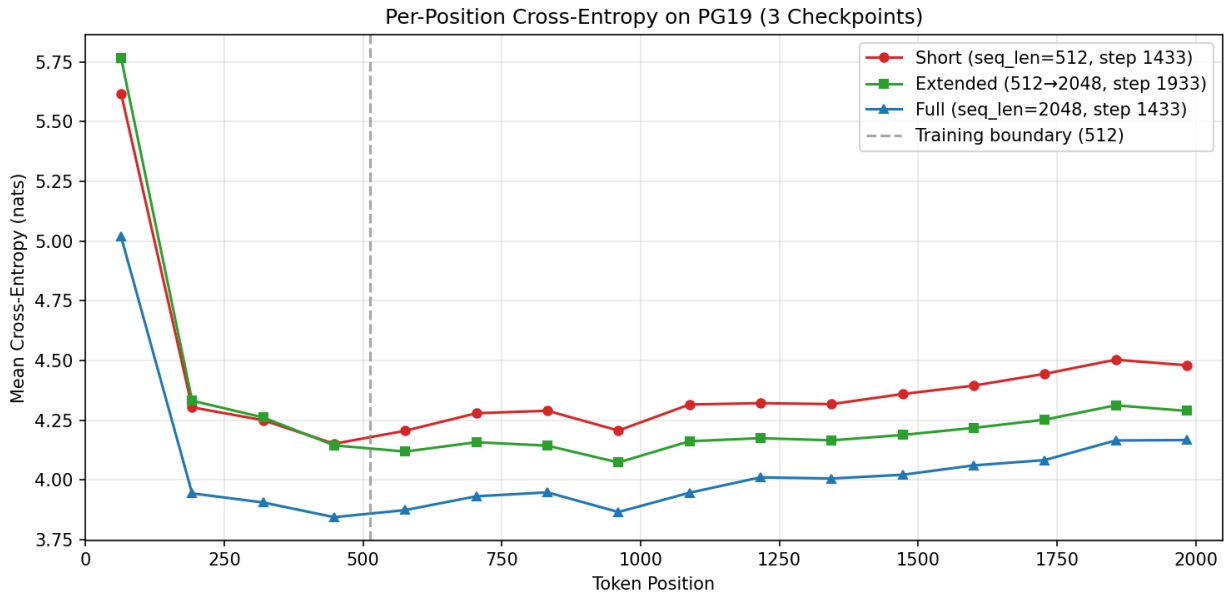

Figure 1: Per-position cross-entropy on PG19 for three checkpoints. Dashed vertical line marks the 512-token training boundary.

The short checkpoint (red) shows a gradual upward drift in cross-entropy beyond position 512, rising from ∼4.15 at positions 384–512 to ∼4.50 at positions 1920–2048—a +0.35 nats degradation. This is not catastrophic but is consistent: positions the model never trained on are handled worse.

The extended checkpoint (green) improves over short beyond position 512 (mean CE 4.19 vs 4.34, a 0.15 nats improvement), confirming that context extension mechanically worked.

The full checkpoint (blue) achieves the lowest CE at all positions, with a consistent advantage that narrows from ∼0.30–0.40 nats at early positions to ∼0.15 nats toward the end of the context—including positions 0–512 where both models had training data.

1.3.2 Recovery Evolution

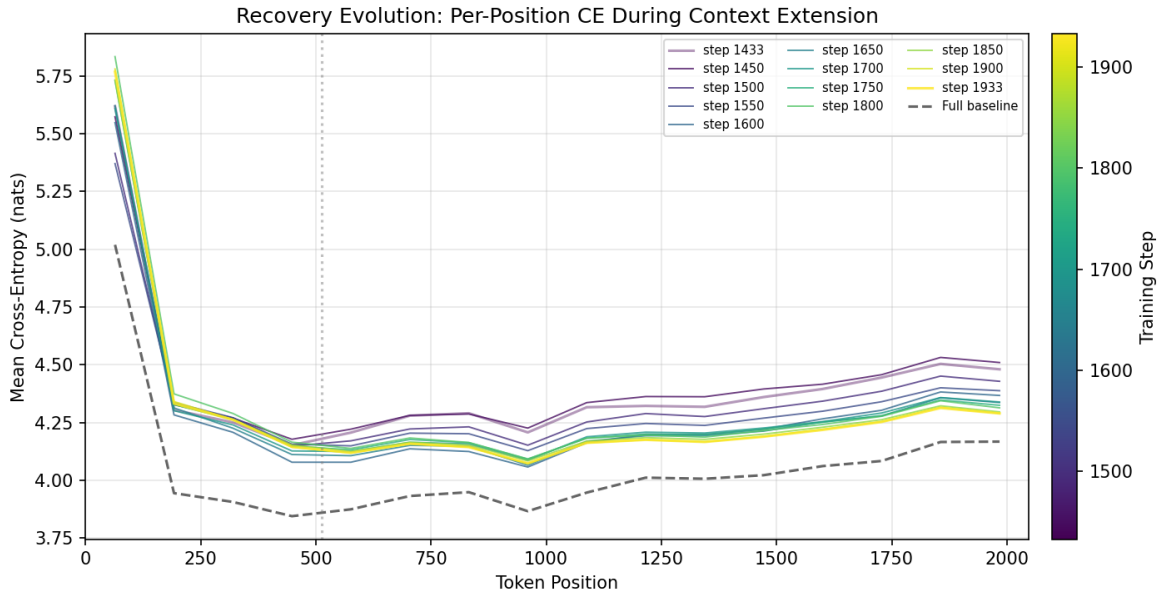

Figure 2: Per-position cross-entropy at each extension checkpoint (steps 1433–1933). Dashed black line is the full-from-scratch baseline.

The model improves most rapidly in the first ∼150 extension steps (1433→1600). Beyond step 1600, PG19 CE plateaus and oscillates mildly, with the final value (4.298) close to the step-1600 minimum (4.288). The largest gains are at positions 512–1024, where the model transitions from untrained to trained positions.

1.3.3 Training Curves

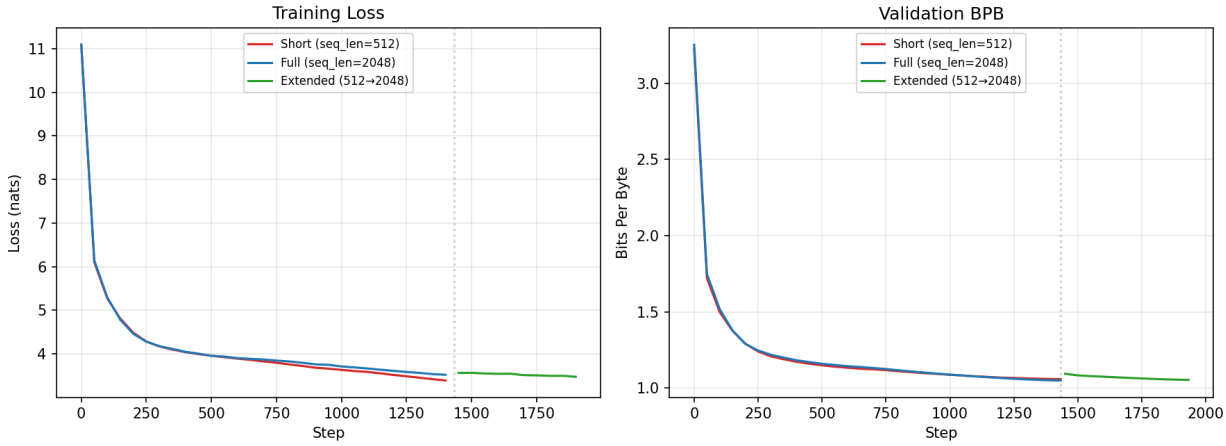

Figure 3: Training loss (left) and validation BPB (right) from W&B. Dotted vertical line marks step 1433 where extension begins.

Short and full training curves overlap closely—both reach similar loss levels despite different sequence lengths. The extension phase (green, steps 1433–1933) shows a mild increase in training loss at resume (from the 4× context jump and LR rewarm), followed by steady recovery. Validation BPB during extension decreases monotonically toward the full baseline.

1.3.4 BPB and CORE Recovery

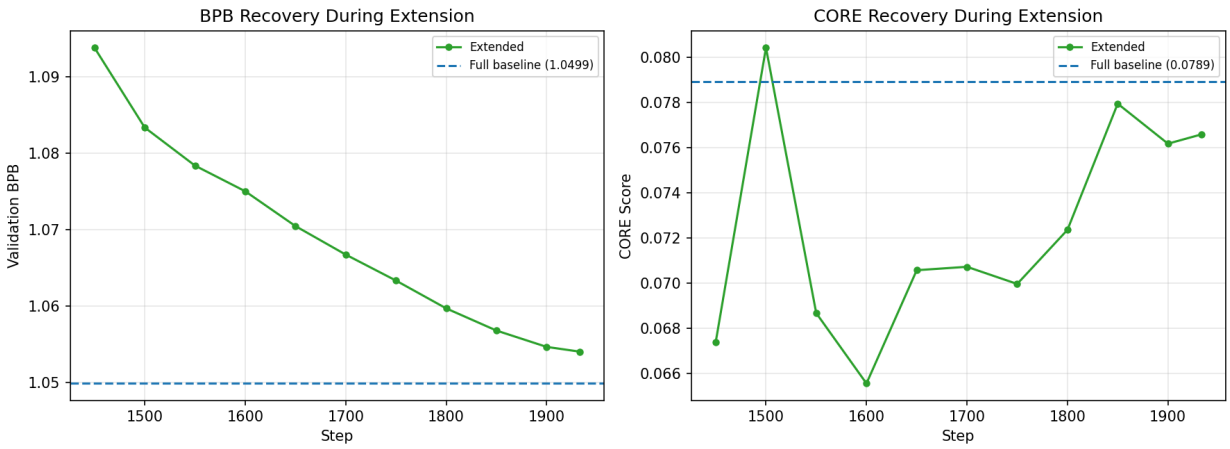

Figure 4: Validation BPB (left) and CORE score (right) during extension. Dashed blue line is the full-from-scratch baseline.

BPB recovery is smooth and monotonic, approaching the full baseline (1.0540 vs 1.0499 at convergence). CORE recovery is noisier—expected at this model scale where individual task scores fluctuate—but trends upward, reaching 0.0766 vs the full baseline's 0.0789.

1.3.5 Summary

Metric	Short (1433)	Extended (1933)	Full (1433)
Val BPB	1.0586	1.0540	1.0499
Train Loss	3.3921	3.4769	3.5236
PG19 Aggregate CE	4.4029	4.2980	4.0500
PG19 Aggregate PPL	81.69	73.55	57.40
CE (pos 0–512)	4.5811	4.6267	4.1785
CE (pos 512–2048)	4.3435	4.1884	4.0071
CORE	N/A	0.0766	0.0789

Table 1: Summary of all metrics. Short CORE is N/A because RoPE at seq_len=512 cannot process 2048-token CORE prompts.

1.3.6 Cost Report

Run	GPU-Hours	Description
Pipeline (baseline)	∼1.51	3 stages + 3 evals
Pipeline (CORE recovery)	∼1.34	10 CORE evals at intermediate steps
Debug / failed attempts	∼1.83	13 smaller runs
Total	**4.67**	**15 Modal app runs**

Table 2: GPU-hours breakdown. Estimated cost: $29.77 at A100-80GB rates ($6.37/hr).

The two successful pipeline runs used ∼2.85 GPU-hours total. Debug runs account for 39% of total cost—expected for a first-time pipeline with multiple failure modes (OOM on A100-40GB, RoPE overflow for short-context CORE, nanochat import paths, BFloat16 autocast).

1.4 Discussion

Context extension mechanically works: the model recovers from the 4× context jump within ∼150 steps, with BPB and CORE approaching the full-from-scratch baseline. However, a persistent PG19 perplexity gap (28%) at all positions—including those seen during short training—indicates forgetting that nanochat's pipeline cannot mitigate. Extension is a viable cheap path for general capability but not for long-context quality.

1.4.1 Recovery Dynamics

On resume at step 1433, the model experiences two simultaneous shocks: a 4× context length jump (512→2048) and a learning rate rewarm (from ≈0 to ≈0.52 of peak). The training loss spikes mildly (Figure 3), then recovers steadily.

The recovery is front-loaded. Per-position cross-entropy improves most rapidly in the first ∼150 steps (1433→1600), with the largest gains at positions 512–1024 where the model encounters truly novel positions for the first time (Figure 2). BPB recovery is smooth and monotonic throughout (Figure 4, left). CORE recovery is noisier but trends upward (Figure 4, right)—the noise is expected at this model scale, where individual task scores fluctuate between checkpoints.

1.4.2 Does the Cheap Path Work?

The cheap path (512→2048 extension) nearly matches the expensive path (2048 from scratch) on general-purpose metrics:

- **BPB**: 1.0540 vs 1.0499—a 0.0042 gap. Close enough for practical purposes.
- **CORE**: 0.0766 vs 0.0789—within noise at this model scale.

However, on the positional perplexity evaluation designed specifically for context extension, the gap is substantial: PG19 perplexity of 73.55 vs 57.40 (28%). This gap persists at *all* positions,

including 0–512 where both models had training data (CE 4.63 vs 4.18). Indeed, the extended model is slightly worse than the short-only model at positions 0–512 (CE 4.63 vs 4.58), confirming that extension actively degrades performance at originally trained positions. ProLong [3] identifies this as forgetting: without short-context data mixing during extension, the model partially unlearns what it knew. Nanochat does not support data mixing, so this forgetting is unmitigated.

1.4.3 Compute Efficiency

Stage 2 ran 500 steps at seq_len=2048 vs Stage 3's 1433 steps—roughly 65% fewer long-context training steps. However, including the short-context phase, the progressive path uses more total compute (127.9T vs 94.8T FLOPs). The savings materialize only when extending a pre-existing short-context model, not when both paths start from scratch. Whether the trade-off is worthwhile depends on the use case: for BPB and CORE (general capability), the cheap path is viable; for tasks requiring strong long-context performance, training from scratch is preferable.

Limitations. We tested a single model scale (∼136M parameters), a single jump ratio (4×), and evaluated on a single long-document dataset (PG19). ProLong's 40% short-context data mixing—which would likely close the PG19 gap—is unavailable in nanochat. Results may differ at larger scales, with progressive multi-stage schedules (as in GrowLength), or on other evaluation domains.

Appendix

W&B Runs

- **p3-baseline-short**: https://wandb.ai/seonghyun-ban-uoft/490-autobook-a3/runs/mpaiemt0
- **p3-baseline-extended**: https://wandb.ai/seonghyun-ban-uoft/490-autobook-a3/runs/woxwtud7
- **p3-baseline-full**: https://wandb.ai/seonghyun-ban-uoft/490-autobook-a3/runs/wy8sw0zf

CORE Per-Task Breakdown for Extended Picochat and Full Picochat

Task	Extended (1933)	Full (1433)
CORE (aggregate)	**0.0766**	**0.0789**
hellaswag	0.2731	0.2745
hellaswag_zeroshot	0.2810	0.2767
arc_easy	0.4289	0.4314
copa	0.6100	0.5500
piqa	0.5871	0.5996
boolq	0.4529	0.5899
winograd	0.5311	0.5018
winogrande	0.5170	0.5043
commonsense_qa	0.2703	0.2817
openbook_qa	0.2740	0.2540
lambada_openai	0.1820	0.1814
bigbench_qa_wikidata	0.1469	0.1284
bigbench_cs_algorithms	0.3977	0.3886
bigbench_dyck_languages	0.1210	0.0530
agi_eval_lsat_ar	0.2826	0.2435
arc_challenge	0.2304	0.2457
bigbench_language_id	0.2574	0.2526
bigbench_operators	0.0667	0.0667
squad	0.0035	0.0033
coqa	0.0256	0.0370
jeopardy	0.0005	0.0019
bigbench_repeat_copy	0.0000	0.0000

Table 3: CORE per-task accuracy. Most scores are within noise. The largest differences (boolq, copa) go in opposite directions, consistent with variance rather than systematic capability gaps.

References

References

[1] H. Jin, X. Han, J. Yang, Z. Jiang, C.-Y. Chang, and X. Hu. GrowLength: Accelerating LLMs Pretraining by Progressively Growing Training Length. *arXiv:2310.00576*, 2023.

[2] T. Zhu, Q. Liu, H. Wang, S. Chen, X. Gu, T. Pang, and M.-Y. Kan. SkyLadder: Better and Faster Pretraining via Context Window Scheduling. *NeurIPS*, 2025. arXiv:2503.15450.

[3] T. Gao, A. Wettig, H. Yen, and D. Chen. How to Train Long-Context Language Models (Effectively). *ACL*, 2025. arXiv:2410.02660.

[4] S. Salhan, R. Diehl Martinez, Z. Goriely, and P. Buttery. What is the Best Sequence Length for BabyLM? *BabyLM Workshop @ EMNLP*, 2025. arXiv:2510.19493.

[5] J. Hoffmann, S. Borgeaud, A. Mensch, et al. Training Compute-Optimal Large Language Models. *NeurIPS*, 2022. arXiv:2203.15556.

[6] S. Chen, S. Wong, L. Chen, and Y. Tian. Extending Context Window of Large Language Models via Positional Interpolation. *arXiv:2306.15595*, 2023.

[7] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. YaRN: Efficient Context Window Extension of Large Language Models. *ICLR*, 2024. arXiv:2309.00071.

[8] Y. Ding, L. Zhang, C. Jia, et al. LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens. *arXiv:2402.13753*, 2024.